Document 16 — System Design & Architecture

> This is for self-study and live mentoring. Full plan: CURRICULUM.md

Prerequisites

14_debugging_lab (you need the failure vocabulary from it), 15_five_projects_index

How to Read & Practice This Document

- **What:** turning an unclear business request into a design you can actually defend, and defending your trade-offs when pushed.
- **Why:** this is the skill that separates "can build a feature" from "can be trusted to own a whole system" — and it's what a senior-level interview actually tests.
- **When:** any time you're handed a vague request instead of a clear spec.
- **How to practice:** for each prompt below, write your design **before** discussing it with me — I review it like a senior architect afterward, I don't hand you the design first.

The Story — what this document is actually building

Every document before this one hands you a fairly clear spec to build against. This one hands you almost nothing — sometimes just one sentence, sometimes a few people wanting different things — and asks you to turn that into a real system design, the way it actually happens at a job. Nobody hands a new hire a finished spec for an AI feature. They say "can we build a bot that answers customer questions," or three stakeholders each want something slightly different, and turning that fog into an actual architecture — which agents, what they do, what happens when something breaks — is the job.

That's why the 3 exercises below escalate the way they do:

- **Basic** — one clean sentence, one agent. Practice not overbuilding, and asking the couple of questions that actually matter first.
- **Intermediate** — a clean request needing several agents in a specific order. Same matching skill, plus picking the right handoff pattern and deciding what happens when a step fails.
- **Real-world** — the "clean" part is gone. Different people want different, conflicting things. Spotting the conflicts and asking about them comes before any design.

Everything you produce maps onto the "What You'll Design (for each prompt)" checklist below — same categories every time, just harder to fill in cleanly as requirements get messier. And the whole thing is rehearsal for "Interview Topics Preview" further down: a real system-design interview is a vague prompt, your design, pushback, and you defending or revising it. Do all three exercises, and that format stops being intimidating.

Jump to: Core Concepts · Practice Exercises

Core Concepts (read this first — everything you need is here)

Unclear requirements are the real test

A vague request ("build a customer support bot") is not a spec with a few details missing. It is the real starting point. A **requirement** is one thing the system must do or must be (for example "answers in under 5 seconds" or "never shows one customer another customer's data"). **Why "just fill in the blanks yourself" is the wrong habit:** the real skill being tested is *asking the right questions before you design anything*. What happens if the bot gets an answer wrong? How many people will use it? What data is it allowed to see? If you silently guess these answers, your design breaks in hidden ways. If you write your guesses down as **assumptions**, everyone can see them, and anyone can say "no, that's wrong" before you build. **When to ask vs. when to assume:** ask when the answer would change the design (a wrong answer costs money, or the user count is 50 vs. 500,000). Assume, and write it down, when the answer changes nothing big, or when nobody is available to answer. **How it works:** read the request, list every question whose answer would change a component, ask the most important 3-5, and write the rest as assumptions at the top of your design.

The questions that change a design most often, and what each answer decides:

Question to ask	Why it matters	What a different answer changes	Small example
What happens if it gets it wrong?	Decides how much checking and human review you need	Low risk: answer directly. High risk: add a checker step or a human handoff	A wrong movie tip is fine; a wrong refund amount is not
How many users, and how many requests at the busiest time?	Decides if one server is enough	50 users: one process. 500,000 users: a queue and many workers	"5 requests a minute" vs. "2,000 requests a minute"
What data can it read, and who owns that data?	Decides the tools, RAG, and security	Public docs: simple RAG. Private customer records: login checks and per-user filters	A bot over the public FAQ vs. a bot over order history
How fast must it answer?	Decides if work happens now or in the	"Live chat": fast model, streaming. "Email reply by	Chat reply in 3 s vs. a report ready in 10 min

Question to ask	Why it matters	What a different answer changes	Small example
	background	tomorrow": a background job is fine	
What is the budget per month?	Decides model size and how many LLM calls per request	Tight budget: one small model call. Big budget: several agents	$50/month vs. $5,000/month
Who decides when two people want different things?	Stops you designing for the wrong person	You design for the decision-maker's goal, and note the others	Sales wants "always answer"; Legal wants "refuse if unsure"

Real-world examples, by situation:

A one-sentence request at a new job:

```
Request: "Build a bot that answers questions about our HR policies."

My questions:            1) Who uses it — all 300 staff, or HR only?
                         2) Can a wrong answer cause harm (leave, salary)?
                         3) Are the policies in one place (PDFs? a wiki?)
Written assumptions:     - ~300 staff, max ~20 questions per hour
                         - Wrong salary/leave answers are harmful
                           -> always show the source
                         - Policies are ~40 PDFs, updated twice a year
```

Three short questions already decide a lot: RAG over PDFs (Doc08), show sources, and no queue needed at 20 questions per hour.

Two people want different things:

```
Marketing: "The bot should answer everything, never say 'I don't know'."
Support:   "The bot must hand off to a human when it is not sure."
Conflict:  always answer  <->  hand off when unsure
Question:  "Which is worse for us: a wrong answer,
            or a slower answer from a human?"
```

You do not pick a side on your own. You name the conflict and ask the one question that settles it.

Nobody is available to answer:

```
ASSUMPTION A1: traffic is under 100 requests/minute.
  If this is wrong: add a queue in front of the agent (see "Scaling plan").
```

You still design, but the assumption is visible, and you already say what changes if it is wrong.

Where you'll meet it: in all three exercises below. The Real-world exercise is built around it. Doc17 system-design questions start exactly like this: a vague prompt, and the interviewer waits to see if you ask questions. On the road to multi-agent systems, the same questions decide how many agents you need in Doc11, and in Project 4 and Project 5: "must the article be fact-checked before publish?" is a requirement question, and its answer adds a whole fact-check agent. In real jobs you meet it in every new feature: a support desk, a RAG search over company docs, a nightly report job.

A common mistake: starting to draw boxes (agents, databases) in the first minute, and never writing down what you assumed. What it causes: you build a large multi-agent system for 20 users, or a simple chatbot that is not allowed to see the data it needs. Later, when someone says "that's not what we meant", nobody can see which guess was wrong. How to spot it: look at your design. If there is no "Assumptions" list at the top, and no question you asked first, you have made this mistake.

Quick cheat sheet:

- Ask before you design: risk of a wrong answer, users, data access, speed, budget.
- Ask only the questions whose answers would change a component.
- Write every guess as a numbered assumption, with "if this is wrong, I change X".
- When two people conflict, name the conflict and ask who decides. Don't pick silently.
- A visible wrong assumption is easy to fix. A hidden one breaks the design later.

Requirements → architecture is matching, not inspiration

Architecture means the main parts of a system (agents, tools, databases, APIs, queues) and how they connect. Every one of those parts should trace back to a specific requirement, not to "this pattern is popular right now". **Why this matters:** each extra part costs something. It adds money (more LLM calls), delay, code to maintain, and new ways to fail (see Doc14). A part with no requirement behind it gives you all that cost and no benefit. **When to add a part:** when you can point to the requirement it satisfies, and a simpler option cannot satisfy it. **When NOT to:** when your only reason is "we might need it later" or "real systems have one". **How it works:** make a two-column table. Left column: requirements. Right column: the part that satisfies each one. Then check both directions. Every requirement needs a part (or it is not handled). Every part needs a requirement (or it is extra and should be removed).

Common requirements and the part they usually match to:

Requirement (situation)	What to add	Why	Small example
Answers must come from company documents	RAG: a vector store plus a retriever tool (Doc08)	The model does not know your private docs	"What is our refund window?" answered from the policy PDF

Requirement (situation)	What to add	Why	Small example
Task has fixed, ordered phases	Sequential pipeline (Doc11)	Each phase has one clear job	Research → draft → fact-check → publish
Right specialist is not known until the request arrives	Supervisor agent that routes	Only a router can decide on the fly	A PR review: security, style, or tests? (Project 13)
Output quality must be checked before use	Generator → critic loop, with a max-loop limit	The creator cannot reliably check itself	Fact-checker sends the draft back, max 2 times
Must remember past conversations	A database for chat history or long-term memory	The model forgets everything between calls	A user returns next week and the bot knows their order
Other systems must call it	An HTTP API, for example FastAPI (Doc12)	Other apps need a stable way in	A website sends `POST /ask`
A wrong action is costly	A human-approval step before the action	A person catches mistakes the model makes	Refunds over $100 wait for a human "approve"
No requirement points here	**Nothing — leave it out**	Every part costs money, delay, and failure modes	No cache when there are 20 users

Real-world examples, by situation:

Checking a design in both directions (a small support bot):

```
Requirement                                  -> Part
R1 answer from product docs                  -> RAG retriever tool
R2 hand off to a human if unsure             -> confidence check
                                                + "handoff" tool
R3 ~50 questions per day                     -> one agent, one small server
(no requirement)                             <- Redis cache       => REMOVE
(no requirement)                             <- 3 extra agents    => REMOVE
```

Two parts had no requirement behind them, so they come out. The final design is one agent with two tools.

A requirement that really does need several agents:

```
"Research a topic, draft an article, fact-check it,
 publish only if the check passes."
```

```
[Researcher] -> [Writer] -> [Fact-checker] --pass--> [Publisher]
                  ^              |
                  +----fail------+   (max 2 revisions, then stop and report)
```

Each box matches one phrase of the request. The loop exists because of "only publish if the check passes". The max-2 limit exists because an endless loop is a failure type from Doc14.

Same wish, different requirement, different part:

```
"Reports should be fast."
 - Means "open the dashboard in 1 s"
     -> pre-compute reports in a nightly job
 - Means "a new report ready in 1 min"
     -> a background worker, and notify by email
```

The words were the same. The real requirement was different, so the matching part was different.

Where you'll meet it: in every exercise below: the "What You'll Design" checklist is a list of parts, and each one needs a requirement next to it. Doc11's patterns (sequential, supervisor, parallel, critic loop) are the menu you match against. On the way to real multi-agent systems, Project 4 (5 agents), Project 11, and Project 13 are all good practice: for each agent, say which requirement it exists for. In Doc18, you should be able to defend every agent, table, and endpoint this way.

A common mistake: copying a design you saw somewhere ("a supervisor with 5 specialists") and then looking for requirements to justify it. What it causes: 3-4 times the cost and delay of a single agent (Doc11 explains why), more places to fail, and a system that is hard to debug. How to spot it: cover the right column of your requirement table and ask "why does this part exist?" If your answer is "it's best practice" or "for the future", not a requirement, the part is extra.

Quick cheat sheet:

- Start simple: one agent with tools. Add parts only when a requirement forces it.
- Every part must name its requirement. Every requirement must name its part.
- Doc11's patterns are a menu: match the task shape to the pattern, not the reverse.
- "We might need it later" is not a requirement. Write it as a future step instead.
- Every loop needs a stop limit, and every risky action needs a check.

Non-functional requirements shape the design just as much as functional ones

A **functional** requirement says *what* the system does ("answer questions about orders"). A **non-functional** requirement says *how well* it must do it: how fast (**latency**, the wait time for one answer), how many at once (**throughput** or scale), how reliable (**availability**, for example "works 99.9% of the time"), how much it may cost, and how secure it must be. The "what" is usually clear from the

request. The "how well" is what actually decides the architecture. A support bot for 50 staff inside a company, and a support bot for 500,000 members of the public, are the *exact same functional request*. The correct designs are completely different. **Why beginners skip this and experienced engineers don't:** the functional request alone cannot tell you if you need a queue, a background job, caching, a smaller model, or more than one agent. Only numbers for speed, scale, cost, and reliability can tell you that. **When to use it:** in every design. If nobody gives you numbers, ask for them, or write your own as assumptions. **How it works:** turn each vague wish into a number ("fast" becomes "under 3 seconds for 95% of requests"), then check each part of your design against each number.

Non-functional requirements and how each one changes the design:

Requirement	If it is strict	If it is relaxed	Small example
Latency (how long one answer takes)	Fewer LLM calls per request, a faster model, streaming, parallel agents	Sequential agents and a bigger model are fine	Live chat: < 3 s. Weekly report: 10 min is OK
Scale (requests at the busiest time)	A queue, several workers, rate-limit handling (Doc02 backoff)	One process on one server	2,000/min at a sale vs. 20/hour inside a company
Cost (money per request or per month)	One small-model call, caching of repeated answers, fewer agents	A multi-agent pipeline with a critic loop is fine	$0.001 per answer vs. $0.50 per article
Reliability (it must keep working)	Retries, a fallback model, save progress after each step (checkpoints, Doc09)	A failed run can just be run again	Payments bot vs. an internal "idea generator"
Accuracy / risk	Fact-check agent, citations, human approval	Answer directly	Medical or money advice vs. a recipe suggestion
Security / privacy	Login, per-user data filters, prompt-injection defense (Project 9), audit logs	Public data only, no login	Customer bank details vs. the public FAQ
Monitoring (you must see problems)	Trace every agent step, cost and error dashboards (Doc13)	Basic logs (Doc01's `logger`)	A 24/7 public product vs. a weekend demo

Real-world examples, by situation:

Same function, two scales:

```
Internal HR bot (300 staff, ~20 req/hour):

 [User] -> [FastAPI] -> [Agent] -> [LLM]
                           |
```

```
                      [Vector DB]

Public store bot (500k users, 2,000 req/min at peak):

 [Users] -> [Load balancer] -> [FastAPI x N]
                                    |
                                [Queue] -> [Workers x N] -> [LLM]
                                    |              |
                               [Answer cache]  [Vector DB]
```

The first design is correct for 300 staff. The second would be wasted money there. For 500,000 users, the first design falls over at the first sale.

A latency number removes an agent:

```
Requirement: reply in under 3 seconds.
Plan A: supervisor -> specialist -> critic
        = 3 LLM calls x ~1.5 s = ~4.5 s                        FAIL
Plan B: one agent + tools, answer streamed as it is generated
        = ~1.5 s                                               PASS
```

The number, not taste, decided the design. (Streaming means showing the answer word by word while it is being written, so the user sees something at once.)

A cost number changes the model plan:

```
Budget $300/month, 100,000 questions/month  => max $0.003 per question.
 -> small model for normal questions
 -> big model only when the small one says "not sure" (maybe 5% of questions)
 -> cache the 200 most common questions
```

A reliability number adds checkpoints:

```
"A 6-step report pipeline must finish even if one step fails."
 -> save state after every step (checkpoint),
    retry a failed step 2 times with backoff,
    then resume from the last checkpoint instead of starting again.
```

Where you'll meet it: in the "Scaling plan", "Security", and "Watching/monitoring plan" lines of every design below. Project 5 and Doc12 are the same 5 agents as Project 4, changed only because of non-functional needs (an API, a database, Docker, retries). Doc13 is how you *measure* latency, cost, and quality, and Doc19 is how you keep them on target after launch. In multi-agent systems, these numbers decide the pattern: parallel agents for a tight latency target, fewer agents for a tight budget, checkpoints for a long pipeline that must not lose work.

A common mistake: writing only functional requirements, or writing non-functional ones as words without numbers ("fast", "scalable", "cheap"). What it causes: you cannot choose between designs, because every design is "fast" and "scalable" if nobody defined it. Or you ship a 4-agent pipeline that takes 12 seconds for a live chat. How to spot it: search your requirements for "fast", "scalable", "reliable", "cheap". Each one needs a number and a unit next to it (seconds, requests per minute, dollars per month, % uptime).

Quick cheat sheet:

- Functional = what it does. Non-functional = how fast, how many, how reliable, how cheap, how safe.
- Turn every vague word into a number: "fast" becomes "< 3 s for 95% of requests".
- Count the LLM calls per request: that number drives both latency and cost.
- High scale → queue + workers. Long pipeline → checkpoints. High risk → checker or human.
- Same functional request + different numbers = a different correct design.

Defending a design when it's pushed on

When someone asks "why not X instead?", it does not mean your design is wrong. It is the normal format of a senior-level design review, and of a system-design interview. **Why this skill matters:** a design is only trusted if the person behind it can explain the **trade-off** (what you gain, and what you give up) behind each choice. If you cannot explain why, people assume you guessed. If you refuse to change even when the other person is right, people stop trusting your judgment. **When to hold your position:** when X would break one of *this* system's requirements, and you can name which one. **When to change your mind:** when the pushback shows a requirement you missed, or when X meets the requirements at lower cost or risk. **How to handle it well:** (1) repeat the question so you are sure you understood it, (2) name the requirement that drove your choice, (3) explain what X would cost for *this* system specifically, (4) if X is better, say "you're right, I'll change it" and state what changes in the design. Defending a choice out of habit, no matter how good the pushback is, looks worse than changing your view with a clear reason.

How to answer common pushback:

Situation (the pushback)	What to do	Why	Small example answer
"Why not one agent instead of four?"	Name the requirement that needed splitting	Multi-agent is 3-4× the cost; you must show it is worth it	"The fact-check must be separate from the writer, or the writer checks its own mistakes."
"Why not add a cache?"	Compare with your scale and cost numbers	A cache adds a part that can serve stale answers	"At 20 requests/hour, it saves under $1/month. Not worth it yet. I'll add it above 1,000/hour."

Situation (the pushback)	What to do	Why	Small example answer
"What if the LLM API is down?"	Show the failure path, or admit it is missing	Reviewers test if you thought about Doc14's failure types	"Retry 2× with backoff, then a fallback model, then 'we'll email you' message."
Pushback shows a requirement you missed	Agree and update the design out loud	Changing with a reason shows good judgment	"You're right, that data is private. I'll add per-user filters to the retriever."
"Isn't that over-engineered?"	Trace each part back to its requirement	The requirement table is your evidence	"Each part matches R1-R5. If R4 is not real, I'll remove the queue."
You don't know the answer	Say so, and say how you would find out	Guessing confidently is worse than an honest "I'd measure it"	"I'm not sure the small model is accurate enough. I'd test it on 50 real questions first (Doc13)."

Real-world examples, by situation:

Holding your position, with a reason:

```
Reviewer: "Why a supervisor? A fixed sequence is simpler."
You:      "A fixed sequence works if every PR needs every check.
           Our requirement R2 says docs-only PRs must skip the
           security scan to stay under 30 s. A supervisor can skip
           it; a fixed sequence can't. If R2 goes away, I'd switch
           to a sequence."
```

You named the requirement (R2), the cost of X, and when you *would* switch.

Changing your mind, with a reason:

```
Reviewer: "Your critic loop can run forever if the critic always rejects."
You:      "You're right, there's no limit. I'll add max 2 revisions.
           After that the draft goes to a human with the critic's
           notes. That also caps cost at 3 writer calls."
```

You accepted the point and said exactly what changes in the design.

The trade-off note you prepare before the review:

```
DECISION D3: store chat history in Postgres, not in memory
  Chosen because: R6 "conversations survive a server restart"
  Gave up:        ~20 ms per request, one more service to run
  Rejected:       in-memory list (lost on restart),
```

```
                    Redis only (no long-term reports)
  Would change if: history is not needed after the session ends
```

Writing short decision notes like this before the review means you never have to invent a reason under pressure.

Where you'll meet it: the "Move On When" rule for this document is "defend at least one design choice when pushed". Every exercise here ends with my review and pushback. Doc17 interviews use this exact back-and-forth. In Doc18 and Project 5, you defend the whole production multi-agent system: why five agents, why this shared state, why this retry rule. At a real job, it is the design review before any team builds a new AI feature, like a support desk, a RAG search, or a multi-agent content pipeline.

A common mistake: answering "why not X?" with "because it's best practice" or "because that's how I learned it", and then refusing to move. What it causes: the reviewer cannot tell if you understood the trade-off, and a real flaw they found stays in the design. The opposite mistake is just as bad: changing your design at the first question, even when your original choice was right. How to spot it: look at your answer. Does it name a specific requirement or number from *this* system? If not, it is a reflex, not a defense.

Quick cheat sheet:

- Pushback is normal. It is how reviews and interviews work, not an attack.
- Answer with: the requirement → the trade-off → what X would cost here.
- Say when you *would* switch to X. It shows you understand both options.
- If they are right, agree, and state the exact change to the design.
- Write short decision notes (chosen because / gave up / would change if) before the review.

Go Deeper (Optional)

You don't need any of these to understand the Core Concepts above — use them if you want a second explanation or more detail.

- system-design-primer (GitHub) — the standard general system-design reference; read the "how to approach a system design interview problem" section closely.
- Read Anthropic — Building Effective Agents again — the patterns from Doc11 are your building blocks here, not new theory.

Practice Exercises

Why only 3 here, not 5: this document's real practice is live — you design, I review like a senior architect, we go back and forth. These 3 give you a fixed starting point at increasing difficulty; the

actual depth comes from the conversation after each one, not from more written exercises.

Where your designs live: `16_system_design_architecture/practice/` (`mkdir -p practice`). These exercises are written designs, not code, so each one is a Markdown file:

```
practice/
├── conflicting_requirements_design.md  Real-world
├── multi_agent_pipeline_design.md      Intermediate
└── single_agent_design.md              Basic
```

Why each file exists:

- `conflicting_requirements_design.md` — naming stakeholder conflicts before designing, and turning a hidden decision into a written one.
- `multi_agent_pipeline_design.md` — Project 4's shape designed fresh: phases, pattern, state, and a loop that can't run forever.
- `single_agent_design.md` — sizing a design to a one-sentence request: assumptions first, one agent, a clear human handoff.

Jump to a design prompt: Basic · Intermediate · Real-world

Basic — a simple single-agent design

- **What:** design (on paper) a single-agent customer-support bot, given only: "answer questions about our product docs, hand off to a human if unsure."
- **Why:** this checks whether you can translate a one-sentence request into a real design without over-building it — the temptation at this size is to add unneeded complexity.
- **When you'll hit this for real:** literally the first design conversation of almost any new AI feature at a real job — someone gives you one sentence, you have to ask the right questions and produce a design.
- **How to do it:** write down your assumptions first (what happens on a wrong answer? what volume?), then the design: agent, tools, routing, and where it hands off to a human.
- **Save as:** `practice/single_agent_design.md`.
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Intermediate — a multi-agent pipeline design

- **What:** design a multi-agent content pipeline given: "research a topic, draft an article, fact-check it, only publish if fact-check passes."
- **Why:** this is a direct rehearsal of Project 4's actual shape — designing it fresh, from requirements, is different from following steps that already decided the shape for you.
- **When you'll hit this for real:** any "build me a pipeline" request with clearly ordered phases — this is an extremely common real request shape.

- **How to do it:** identify the phases first, decide which pattern from Doc11 fits (sequential? does fact-check need to loop back to draft?), then design agents/state/routing around that decision.
- **Save as:** `practice/multi_agent_pipeline_design.md`.
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Real-world — unclear, conflicting requirements

- **What:** design a system given unclear, conflicting requirements from different people (a realistic, messy prompt).
- **Why:** real requirements are never as clean as a training exercise — this is where the actual senior skill (asking the right clarifying questions before designing) gets tested for real.
- **When you'll hit this for real:** almost every real system-design conversation you'll ever have — stakeholders rarely agree with each other up front.
- **How to do it:** before designing anything, write down every place the requirements conflict or are unclear, and what question you'd ask to resolve each one — then design around your best guess, flagging the assumption.
- **Solo practice prompt** (live sessions use a fresh one): three people want things from the content pipeline in the Intermediate exercise. **Marketing** wants every article published automatically within minutes. **Legal** wants a human to review every article before it's published, no exceptions. **Finance** wants the API/compute cost capped. Nobody has said what happens when Marketing's speed and Legal's review both apply to the same article.
- **Save as:** `practice/conflicting_requirements_design.md`.
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

What You'll Design (for each prompt)

- Agents (how many, which pattern from Doc11, and why)
- Tools
- State (shared vs. private)
- Database
- APIs
- RAG (if it applies)
- Routing logic
- Security
- Scaling plan
- Watching/monitoring plan

Expected Behavior

Your designs should already account for the failure types from 14_debugging_lab, without being asked to — for example, you should already be asking "what stops this from looping forever" before I have to ask you.

Interview Topics Preview

A full system-design interview run-through — this document *is* the interview-prep format (question → your design → my review → correction), just done at the scale of a whole system.

Move On When

Your designs already account for the failure types from Doc14, without me prompting for them, and you can defend at least one design choice when pushed on it directly. Full details: CURRICULUM.md §4.

Stuck on where to even start a design? Ask for **Hint 1** (what question to ask about the requirements first) or **Hint 2** (the plan, and almost the whole design). Ask for a full design only if you say **"Show me the solution."**